

Université de Lorraine – Master 1 Informatique
Année universitaire 2025-2026

RAPPORT DE PROJET

Projet Métaheuristiques Optimisation de trajectoires

*Optimisation de trajectoires 2D en présence d'obstacles
modélisées par des courbes de Bézier*

Naël BENZAADI & Linda RAHALI

Date de rendu : 5 mai 2026

Résumé

Ce rapport présente *OptiPathFinal* (méthode **Fast&Bezier**), une métaheuristique hybride pour l'optimisation de trajectoires 2D représentées par des courbes de Bézier. L'algorithme combine deux moteurs qui tournent en parallèle dès le démarrage : un L-SHADE à grande population ($N_{P0} = 18d$) pour l'exploration large, et un CMA-ES IPOPOP initialisé par sélection au sein d'un pool de graines diversifiées. Ce pool inclut une graine issue d'un A* sur grille 100×100 , sans qu'elle soit toujours retenue : toutes les graines sont évaluées et les meilleures sont raffinées par une brève recherche locale (1+1)-ES, dont la sortie sert de moyenne initiale à CMA-ES. Une décision de bascule à $t = 25$ s peut désactiver le moteur infaisable si l'autre est faisable, sinon les deux continuent.

Table des matières

1	Introduction	1
2	Problème étudié	1
2.1	Encodage Bézier	1
2.2	Fonction objectif officielle	1
2.3	Évaluation multi-critères et détection de faisabilité	2
2.4	Instances	2
3	État de l’art	2
3.1	Évolution différentielle et L-SHADE	2
3.2	CMA-ES et IPOP	2
3.3	Algorithme A*	3
4	Méthodes d’optimisation retenues	3
4.1	Évolution différentielle adaptative (L-SHADE)	3
4.2	Moteur CMA-ES IPOP avec graines diversifiées	3
4.3	Initialisation – vue d’ensemble	3
4.4	Boucle principale	3
5	Algorithme proposé : OptiPathFinal	4
5.1	Architecture générale	4
5.2	Initialisation	4
5.3	Boucle principale	4
6	Implémentation détaillée de CMA-ES	5
6.1	Décomposition propre et gestion de la complexité	6
6.2	Pool de graines diversifiées	6
6.2.1	Catégories de graines	6
6.2.2	Coût de l’initialisation	6
6.3	Stratégie de restart (IPOP)	6
6.4	Gestion des bornes	7
7	Expériences et résultats	7
7.1	Sources de résultats	7
7.2	Résultats du code rendu	7
7.3	Synthèse des benchmarks historiques sur prob4	8
7.4	Lecture des résultats et limites	8
8	Analyse	9
9	Méthodes explorées mais non retenues	10
10	Conclusion	10

1 Introduction

La planification de trajectoire en présence d'obstacles est un problème fondamental en robotique et en automatique. Lorsque la trajectoire est paramétrée par des courbes de Bézier, le problème se réduit à l'optimisation d'un vecteur de points de contrôle internes sous contraintes continues (obstacles, bornes du domaine, courbure). Le paysage de coût résultant est fortement multimodal : une modification infime d'un point de contrôle peut faire traverser une brèche étroite ou, au contraire, produire une collision coûteuse.

Dans ce projet, le budget d'évaluation est limité à 60 secondes par instance. Le cadre impose une unique classe concurrente étendant `CompetitorProject`, une initialisation réalisée dans `initialization()`, aucune parallélisation et des évaluations exclusivement via l'API `Problem`.

Notre approche, Fast&Bezier (classe `OptiPathFinal`), est hybride et fait tourner en parallèle deux métaheuristiques complémentaires :

- **L-SHADE** avec une grande population ($N_{P0} = 18d$) pour explorer largement l'espace de recherche dès les premières secondes.
- **CMA-ES** avec redémarrage IPOP, dont la moyenne initiale est choisie par évaluation et raffinement local au sein d'un pool de graines diversifiées. Ce pool inclut une graine issue d'un A* sur grille 100×100. Cette graine est utile pour les labyrinthes denses mais elle n'est pas systématiquement retenue, c'est la meilleure du pool après recherche locale qui est sélectionnée.

À $t = 25$ s (mi-budget), un mécanisme de sélection automatique désactive le moteur dont la solution est infaisable si l'autre est déjà faisable. Sinon, les deux continuent. Aucun moteur n'est privilégié a priori, et la décision repose uniquement sur la faisabilité observée.

Ce rapport détaille la formulation du problème (section 2), les méthodes utilisées (section 3), la stratégie mise en œuvre (section 5) et les résultats obtenus (section 7).

2 Problème étudié

2.1 Encodage Bézier

Une courbe de Bézier d'ordre n est définie par $n + 1$ points de contrôle $P_0, \dots, P_n$:

$$B(t) = \sum_{i=0}^n \binom{n}{i} (1-t)^{n-i} t^i P_i, \quad t \in [0, 1]. \quad (1)$$

Dans le sujet, P_0 (départ) et P_n (arrivée) sont imposés par l'instance. L'algorithme optimise donc uniquement les $m = n - 1$ points de contrôle internes, encodés dans un vecteur plat de dimension $d = 2m$:

$$x = (x_1, y_1, \dots, x_m, y_m) \in \mathbb{R}^d.$$

Dans la version rendue, cette extension $\delta = 8,0$ est appliquée au moteur L-SHADE. CMA-ES, lui, est borné au rectangle du problème par clamp explicite.

2.2 Fonction objectif officielle

La qualité d'une solution est calculée par `Problem.evaluate(x)` :

$$f(x) = L(x) + 100 P_{\text{obs}}(x) + P_{\text{courb}}(x) + 100 P_{\text{bord}}(x). \quad (2)$$

Le terme L mesure la longueur approchée de la courbe. P_{obs} pénalise les collisions avec les obstacles et leur proximité. P_{courb} pénalise les changements brusques de direction. P_{bord} pénalise les sorties du domaine autorisé.

2.3 Évaluation multi-critères et détection de faisabilité

La méthode `Problem.evaluateMulti(x)` retourne un tableau de quatre composantes (c_0, c_1, c_2, c_3) correspondant aux quatre termes ci-dessus. `OptiPathFinal` l'utilise à deux fins :

1. **Initialisation A***. Le score de chaque cellule de la grille 100×100 est évalué via `evaluateMulti` pour détecter les zones libres d'obstacles.
2. **Décision de bascule à $t = 25$ s.** La violation d'une solution est définie comme $v(x) = \max(0, c_1) + \max(0, c_3)$. Une solution est dite *faisable* si $v(x) = 0$.

2.4 Instances

Huit instances sont disponibles dans le dossier de rendu. Elles diffèrent par le nombre de points de contrôle internes (de 4 à 16 environ), la taille du domaine, le nombre et la disposition des obstacles.

3 État de l'art

3.1 Évolution différentielle et L-SHADE

L'évolution différentielle (DE) [1] génère de nouveaux candidats par différences vectorielles entre individus de la population. La variante **SHADE** (Success-History based Adaptive DE) [2] introduit une mémoire de taille H stockant les couples (F_k, CR_k) ayant produit des améliorations. À chaque génération, F et CR sont rééchantillonnés depuis cette mémoire (Cauchy pour F , gaussienne pour CR) et la mémoire est mise à jour par la *moyenne de Lehmer* pondérée par les gains de fitness :

$$\mu_L(S) = \frac{\sum_i w_i s_i^2}{\sum_i w_i s_i}, \quad w_i = \frac{\Delta f_i}{\sum_j \Delta f_j}. \quad (3)$$

L-SHADE [3] ajoute une réduction linéaire de la population : la taille N_P décroît de N_{P0} à $N_{P\min}$ au cours du budget, libérant des évaluations pour les meilleures solutions. La mutation `current-to-pbest/1` est :

$$v_i = x_i + F(x_{p\text{best}} - x_i) + F(x_{r_1} - x'_{r_2}), \quad (4)$$

où $x_{p\text{best}}$ est tiré aléatoirement parmi les $\lfloor p \cdot N_P \rfloor$ meilleurs individus ($p = 11\%$ dans notre configuration), et x'_{r_2} peut être issu de la population ou de l'archive externe.

3.2 CMA-ES et IPOP

La **stratégie d'évolution à matrice de covariance** (CMA-ES) [4] adapte à chaque itération la matrice de covariance C d'une distribution gaussienne $\mathcal{N}(m, \sigma^2 C)$. La mise à jour suit les chemins d'évolution p_σ et p_c :

$$m^{(g+1)} = m^{(g)} + c_m \sum_{i=1}^{\mu} w_i (x_{i:\lambda}^{(g)} - m^{(g)}), \quad (5)$$

$$C^{(g+1)} = (1 - c_1 - c_\mu) C^{(g)} + c_1 p_c p_c^\top + c_\mu \sum_{i=1}^{\mu} w_i y_{i:\lambda} y_{i:\lambda}^\top. \quad (6)$$

La stratégie **IPOP** [6] redémarre CMA-ES en doublant la taille de population λ à chaque redémarrage, ce qui permet d'échapper aux bassins locaux après une convergence prématurée.

Dans notre implémentation, une candidate issue d'A* est intégrée au pool de graines, puis la moyenne initiale effective est sélectionnée après évaluation et raffinement local (cf. §5.2). Une pénalité renforcée de faisabilité est appliquée sur les contraintes via `ProgressivePenaltyEval`, et l'échantillonnage par miroir (`MirrorSampling`) est activé.

3.3 Algorithme A*

L'algorithme A* [7] calcule le chemin de coût minimal dans un graphe en maintenant une file de priorité sur $f(n) = g(n) + h(n)$, où $g(n)$ est le coût cumulé depuis la source et $h(n)$ une heuristique admissible (distance euclidienne dans notre cas). Nous l'utilisons sur une grille 100×100 discrétisant le domaine pour identifier un chemin sans obstacle du point de départ au point d'arrivée. Les cellules dont l'évaluation `evaluateMulti` révèle une collision sont marquées comme inaccessibles.

4 Méthodes d'optimisation retenues

`OptiPathFinal` fait tourner deux moteurs en parallèle dès le démarrage : un L-SHADE qui explore largement, et un CMA-ES IPOP dont la moyenne initiale est sélectionnée au sein d'un pool de graines diversifiées. Aucun moteur n'est privilégié a priori. Une bascule conditionnelle peut intervenir à mi-budget si l'un est faisable et l'autre non.

4.1 Évolution différentielle adaptative (L-SHADE)

Le premier moteur est une variante L-SHADE [3] avec mémoire SHADE [2]. À chaque génération, un individu cible $\mathbf{x}_i$ est muté par

$$\mathbf{v}_i = \mathbf{x}_i + F_i(\mathbf{x}_{\text{pbest}} - \mathbf{x}_i) + F_i(\mathbf{x}_{r_1} - \mathbf{x}'_{r_2}), \quad (7)$$

puis combiné par croisement binomial. F_i et CR_i sont auto-adaptés via une mémoire circulaire de succès. La population est initialisée uniformément dans un domaine élargi d'une marge $\delta = 8,0$ autour du rectangle visible. Les hyperparamètres exacts sont donnés en §5.2.

4.2 Moteur CMA-ES IPOP avec graines diversifiées

Le second moteur est un CMA-ES avec restart IPOP, borné au rectangle du problème par clamp explicite. Trois choix d'implémentation sont injectés dans le moteur :

- `MirrorSampling` pour la réduction de variance lors de l'échantillonnage gaussien.
- `ProgressivePenaltyEval` comme évaluateur, qui applique une pénalité renforcée de faisabilité sur les contraintes.
- `IPOPRestart` qui double la taille de population λ à chaque redémarrage (plafonné à 512).

Les principes mathématiques (adaptation de covariance, chemins d'évolution, pas global) sont rappelés en §3 et les détails d'implémentation en §6.

4.3 Initialisation – vue d'ensemble

L'initialisation de CMA-ES est répartie entre `OptiPathFinal` (qui calcule une candidate A* sur grille 100×100) et `CMAESCore.init()` (qui génère un pool de graines diversifiées, y insère la candidate A*, évalue, puis raffine les meilleures par `quickLocalOptimize`). La moyenne initiale effective $\mathbf{m}^{(0)}$ est le meilleur résultat de cette sélection. Le `quickLocalOptimize` est aussi appelé dans une partie des redémarrages IPOP. Détails en §5.2 et §6.2.

4.4 Boucle principale

Chaque appel à `loop()` fait avancer L-SHADE d'une génération puis CMA-ES d'une génération, sans échange entre les deux. À $t = 25$ s une bascule conditionnelle peut désactiver le moteur infaisable si l'autre est faisable (cf. §5, pseudo-code).

5 Algorithme proposé : OptiPathFinal

5.1 Architecture générale

OptiPathFinal orchestre deux moteurs internes – LSHADECore et CMAESBuilder.ipop() – qui tournent en parallèle sans échanger ni population ni meilleur courant. La sous-section suivante détaille leur initialisation et §6 couvre les choix d’implémentation de CMA-ES.

5.2 Initialisation

Bornes de l’espace de recherche. L-SHADE opère sur un espace étendu d’une marge $\delta = 8,0$ autour du domaine visible :

$$\ell_i = \min\{X\} - \delta, \quad u_i = \max\{X\} + \delta \quad (\text{et idem pour } Y),$$

ce qui autorise des détours temporaires hors cadre. CMA-ES, en revanche, est directement borné au rectangle du problème par clamp explicite.

Initialisation de L-SHADE. LSHADECore est instancié avec $N_{P0} = 18d$ (288 pour $d = 16$), $H = 6$ cases mémoire pour les couples (F, CR) , archive externe de taille $2,6 N_{P0}$, $N_{P\min} = 4$, $p = 11\%$ pour la mutation `current-to-pbest/1`, F tiré selon une loi de Cauchy et CR selon une gaussienne. `lshade.init()` initialise la population uniformément dans $[\ell, u]^d$. Elle ne reçoit aucune des graines décrites ci-dessous.

Calcul d’une candidate A* dans OptiPathFinal. `computeAStarMean` construit une grille 100×100 sur le domaine, marque les cellules dont l’évaluation `evaluateMulti` (sur un patron en croix) révèle une collision, puis exécute un A* 8-connexe avec heuristique euclidienne du départ vers l’arrivée. Le chemin trouvé est converti heuristiquement en n_{CP} points de contrôle. Le vecteur résultant est passé à `CMAESBuilder.initMean()` : il n’est pas la moyenne initiale effective de CMA-ES, mais une *candidate* graine ajoutée au pool décrit ci-après.

Sélection de la moyenne initiale dans CMAESCore.init(). Au démarrage du moteur, CMAESCore construit un pool d’environ 260 graines diversifiées (sinusoïdes, motifs en blocs, chemins de bord à y constant, tirages aléatoires, regroupements aux frontières. Détails en §6), insère la candidate A* en tête du pool, puis évalue chaque graine via `ProgressivePenaltyEval`. Les 25 meilleures sont mises en cache pour les redémarrages. Les 5 meilleures sont raffinées par `quickLocalOptimize` (un (1+1)-ES avec règle du 1/5 de succès, 1 000 évaluations chacune). Le meilleur résultat de cette recherche locale devient la moyenne initiale $\mathbf{m}^{(0)}$ effectivement utilisée par CMA-ES. La candidate A* est donc retenue ou écartée selon qu’elle survit à cette compétition. Aucune préférence n’est codée en sa faveur.

5.3 Boucle principale

Étape L-SHADE. `lshade.step()` exécute une génération complète : mutation `current-to-pbest/1`, croisement binomial, sélection gloutonne, mise à jour de la mémoire par moyenne de Lehmer pondérée, et réduction linéaire de population.

Étape CMA-ES. `cmaes.step()` exécute une itération : échantillonnage par miroir, évaluation, mise à jour de $\mathbf{m}$, σ et $\mathbf{C}$. CMAESCore gère ses redémarrages IPOP en interne.

Bascule conditionnelle à $t = 25$ s. Au premier appel de `loop()` après le seuil `FALLBACK_MS=25 000` ms, les violations

$$v(\mathbf{x}) = \max(0, c_1) + \max(0, c_3), \quad (c_0, c_1, c_2, c_3) = \text{evaluateMulti}(\mathbf{x})$$

des meilleures solutions courantes des deux moteurs sont comparées. Si L-SHADE est faisable ($v = 0$) et CMA-ES non, CMA-ES est désactivé. Inversement pour L-SHADE. Si les deux sont faisables ou les deux infaisables, les deux moteurs continuent. La bascule est donc asymétrique et n'intervient qu'une seule fois.

Algorithme 1 OptiPathFinal – vue d'ensemble

```

1: procedure INITIALIZATION
2:   Construire  $\ell, u$  avec marge  $\delta = 8,0$ 
3:   Créer LSHADECore( $d, N_{P0}=18d, H=6, \text{archive}=2,6 N_{P0}, N_{P\min}=4, p=0,11$ )
4:   lshade.init() ▷ population uniforme dans  $[\ell, u]^d$ 
5:    $\mathbf{a} \leftarrow \text{computeAStarMean}(\text{grille } 100 \times 100, \text{A}^* \text{ 8-connexe})$ 
6:   Créer CMAESBuilder.ipop() avec ProgressivePenaltyEval, MirrorSampling, initMean=  $\mathbf{a}$ 
7:   cmaes.init() ▷ voir bloc ci-dessous
8: end procedure
9: procedure CMAES.INIT
10:  Générer pool  $\mathcal{S}$  de  $\sim 260$  graines diversifiées
11:   $\mathcal{S} \leftarrow \{\mathbf{a}\} \cup \mathcal{S}$  ▷ candidate  $\text{A}^*$  en tête
12:  Évaluer  $\mathcal{S}$  via ProgressivePenaltyEval
13:  Cacher les 25 meilleures pour les redémarrages
14:  Pour les 5 meilleures,  $\mathbf{s}_k \leftarrow \text{quickLocalOptimize}(\mathbf{s}_k, 1000)$ 
15:   $\mathbf{m}^{(0)} \leftarrow \arg \min_k f(\mathbf{s}_k)$ 
16:  Configurer CMA-ES depuis  $\mathbf{m}^{(0)}$ 
17: end procedure
18: procedure LOOP
19:   if  $t \geq 25$  s and bascule pas encore prise then
20:      $v_L \leftarrow v(\text{lshade.getBestX}())$ 
21:      $v_C \leftarrow v(\text{cmaes.getBestX}())$ 
22:     if  $v_L = 0$  and  $v_C > 0$  then
23:       désactiver CMA-ES
24:     else if  $v_C = 0$  and  $v_L > 0$  then
25:       désactiver L-SHADE
26:     end if
27:   end if
28:   if lshadeActive then lshade.step()
29:   end if
30:   if cmaesActive then
31:     cmaes.step()
32:   end if
33: end procedure

```

6 Implémentation détaillée de CMA-ES

Cette section détaille les choix d'implémentation et les améliorations apportées à CMA-ES, qui constituent la contribution principale de ce travail.

6.1 Décomposition propre et gestion de la complexité

L'eigendecomposition de $\mathbf{C}$ (nécessaire pour l'échantillonnage et le calcul de $\mathbf{C}^{-1/2}$) utilise les algorithmes `tred2` (réduction de Householder) et `trql2` (QR itératif) issus de la bibliothèque JAMA (domaine public). Pour éviter un coût $O(d^3)$ à chaque génération, la décomposition n'est recalculée que toutes les $\max\left(1, \left\lfloor \frac{1}{10 \cdot d \cdot (c_1 + c_\mu)} \right\rfloor\right)$ générations, conformément à la formule du tutoriel CMA-ES [5].

6.2 Pool de graines diversifiées

Le mécanisme global de sélection (~ 260 graines $\rightarrow$ top-25 cache $\rightarrow$ top-5 + `quickLocalOptimize` $\rightarrow \mathbf{m}^{(0)}$) est décrit en §5.2. Cette sous-section en complète les *catégories de graines* et la *discussion de coût*.

6.2.1 Catégories de graines

Cinq familles sont générées dans `generateDiverseSeeds()`, pour un total d'environ 260 graines :

1. **Sinusoïdales** (≈ 160 graines) : pour chaque combinaison de période $p \in \{0,5, 1,0, 1,5, 2,0, 2,5\}$, phase $\phi \in \{0, \pi/4, \dots, 7\pi/4\}$ et amplitude $A \in \{0,35, 0,65, 0,85, 0,95\}$, on construit un vecteur dont les composantes y oscillent autour de l'interpolation linéaire départ–arrivée.
2. **Motifs en blocs** : pour chaque entier $n_{\text{up}} \in \{1, \dots, \lfloor n_{\text{CP}}/2 \rfloor\}$, deux graines miroirs sont générées dont les ordonnées alternent entre la partie haute et la partie basse du domaine par paquets de n_{up} points consécutifs.
3. **Chemins de bord** (3 graines) : trois trajectoires à y constant, respectivement près du bord inférieur ($y = \text{lb}_Y + 1,5$), près du bord supérieur ($y = \text{ub}_Y - 1,5$) et au centre vertical ($y = (\text{lb}_Y + \text{ub}_Y)/2$).
4. **Aléatoires** (20 graines) : tirées uniformément dans les bornes.
5. **Regroupements aux frontières** (≈ 63 graines) : les abscisses sont forcées à $x_{\min}$ ou $x_{\max}$ (en alternance ou regroupées), avec différents profils d'ordonnées, pour exploiter la vitesse paramétrique des courbes de Bézier dans les domaines très contraints.

6.2.2 Coût de l'initialisation

Le `quickLocalOptimize` appliqué aux 5 meilleures graines est un (1+1)-ES avec règle du 1/5 de succès, pas initial $\sigma_{\text{LS}} = \sigma_0/2$, et 1 000 évaluations par graine. Le surcoût total de l'initialisation est donc d'environ $260 + 5 \times 1\,000 \approx 5\,260$ évaluations, négligeable devant le nombre total d'évaluations effectuées en 60 s sur les instances testées.

6.3 Stratégie de restart (IPOP)

Lorsque la convergence est détectée, l'algorithme effectue un *restart*. Trois critères sont vérifiés à chaque génération :

1. **Stagnation de la *fitness*** : le meilleur score de la génération courante ne dépasse pas celui de la génération précédente d'au moins 10^{-12} . Le compteur de stagnation s'incrémente et déclenche un restart après $10 + \lfloor 30d/\lambda \rfloor$ générations consécutives sans progrès.
2. **Effondrement du pas effectif** : $\sigma \cdot \max(\text{diagD}) < 10^{-12}$, indiquant que la distribution de recherche est devenue trop étroite pour progresser.
3. **Dégénérescence de $\mathbf{C}$** : le rapport $\max(\text{diagD})/\min(\text{diagD})$ des racines des valeurs propres dépasse 10^7 , signalant un conditionnement numérique pathologique.

La stratégie de restart retenue est **IPOP** : la taille de population est doublée à chaque redémarrage ($\lambda_{\text{new}} = 2\lambda_{\text{old}}$), plafonnée à 512, pour explorer des bassins d'attraction de plus en plus larges.

Le nouveau point de départ (`getDefaultRestartMean`) est tiré selon trois règles disjointes : 40% une graine prise dans le cache des 25 meilleures, raffinée par `quickLocalOptimize` sur 500 évaluations.

30% le meilleur point trouvé jusqu'à présent, perturbé par un bruit gaussien d'amplitude 15 % de l'étendue de chaque dimension.

30% le meilleur parmi 8 candidats (4 aléatoires et 4 regroupements aux frontières).

La règle à 40 % explique pourquoi `quickLocalOptimize` n'est pas confiné à l'initialisation : il est aussi appelé pendant les redémarrages.

6.4 Gestion des bornes

Toute composante sortant des bornes $[lb_i, ub_i]$ est **tronquée** à la borne la plus proche (*box constraint handling*). Pour CMA-ES, le clamp empêche directement les points de contrôle de sortir du rectangle du problème. Pour L-SHADE, les bornes élargies restent accompagnées de la pénalité de sortie (P_{bord}).

7 Expériences et résultats

7.1 Sources de résultats

Cette section combine deux jeux de résultats de natures différentes, qu'il faut distinguer :

1. **Code rendu (OptiPathFinal, version finale)**. Résultats obtenus sur les huit instances avec le programme `Main.java` fourni, sans modification du framework. Les valeurs ci-dessous proviennent d'un *lancement unique* par instance complété par quelques tests informels. Nous n'avons pas pu lancer un protocole à 10 exécutions indépendantes dans les délais. Les plages reflètent donc une variabilité observée plutôt qu'une statistique propre.
2. **Benchmarks historiques (/ancien)**. Comparaisons à 3 exécutions indépendantes par couple (algorithme, instance), sur `prob1`–`prob4` uniquement, conduites pendant la phase d'exploration. Elles servent à comparer des familles d'algorithmes et à motiver les choix de la version rendue, mais elles ne constituent pas une évaluation de la soumission finale : les algorithmes comparés sont des moteurs isolés (CMA-ES, DE, SHADE, ...) et non l'hybridation `OptiPathFinal`.

7.2 Résultats du code rendu

TABLE 1 – Code rendu `OptiPathFinal` – lancement unique sur les huit instances. Les plages indiquent la variabilité observée lors de tests informels (3 à 5 reprises au plus). La colonne « moteur dominant » est qualitative et n'est pas une mesure objective.

Instance	Score observé	Faisable	Moteur dominant (qualitatif)
<code>prob1</code>	37,42	✓	L-SHADE / CMA-ES
<code>prob2</code>	31,42	✓	L-SHADE / CMA-ES
<code>prob3</code>	35,57 – 38,80	✓	L-SHADE / CMA-ES
<code>prob4</code>	1562 – 1862	×	–
<code>prob5</code>	29,43	✓	L-SHADE / CMA-ES
<code>prob6</code>	88,25 – 88,31	✓	CMA-ES
<code>prob7</code>	62,78 – 62,93	✓	L-SHADE
<code>prob8</code>	28,46 – 29,20	✓	L-SHADE / CMA-ES

Sept instances sur huit sont résolues avec une solution faisable. `prob4` reste partiellement infaisable. Les scores observés (1562–1862) restent du même ordre que ceux des moteurs isolés en configuration « marges étendues » (cf. tableau 2).

7.3 Synthèse des benchmarks historiques sur prob4

Sur **prob1–prob3**, presque toutes les méthodes testées convergent vers des scores très proches : ~ 36 – 37 pour **prob1**, $\sim 31,48$ pour **prob2**, ~ 28 – 30 pour **prob3**. Ces trois instances discriminent peu les algorithmes. **prob4** en revanche révèle l’effet de l’extension des bornes hors du rectangle visible. Le tableau 2 regroupe les moyennes et les meilleurs scores sur 3 runs pour les deux moteurs effectivement utilisés dans le code rendu (L-SHADE et CMA-ES), en comparant la configuration *baseline* (pas de marge) à la configuration *margin8* (marge $\delta = 8$, identique à celle utilisée côté L-SHADE dans le code rendu).

TABLE 2 – **prob4** – benchmarks historiques sur 3 exécutions indépendantes. Moyennes et meilleurs scores en fonction de l’extension des bornes. Source : `results/bench_ExtendedBounds.csv`.

Configuration	Moteur	Moyenne (3 runs)	Meilleur (3 runs)
baseline	L-SHADE	5 418,9	5 418,9
baseline	CMA-ES	6 026,9	5 809,0
margin8	L-SHADE	1 517,3	1 516,9
margin8	CMA-ES	1 586,7	1 517,4

L’extension des bornes divise par 3 à 4 la moyenne de **prob4** pour les deux moteurs et fait passer le meilleur score sous la barre de 1 520. Le résultat le plus marquant concerne L-SHADE en *margin8* : les trois runs atteignent 1 516,9, 1 518,1 et 1 516,9, soit une dispersion inférieure à 1,3 point sur 1 517 – une stabilité inhabituelle sur cette instance. CMA-ES en *margin8* obtient un meilleur score comparable (1 517,4) mais avec une dispersion plus large (un run à 1 705 tire la moyenne vers le haut). Cette stabilité de L-SHADE sous *margin8* a directement motivé le choix d’appliquer la marge $\delta = 8$ à L-SHADE dans la version rendue.

Sur **prob1–prob3**, l’effet de *margin8* est mineur : les scores restent dans la même fourchette que la baseline, voire légèrement moins bons sur quelques runs (par exemple L-SHADE **prob3** passe de $\sim 29,7$ à $\sim 32,0$ en moyenne avec *margin8*). L’extension n’est donc pas universellement bénéfique. C’est sur les instances très contraintes qu’elle paie.

Précaution. Ces chiffres concernent un moteur isolé exécuté seul pendant 60 s, pas l’hybridation `OptiPathFinal`. La stabilité observée sur L-SHADE *margin8* **prob4** dans ce benchmark historique ne se transpose pas mécaniquement à un score garanti pour la soumission finale, qui partage le budget entre L-SHADE et CMA-ES et applique une bascule conditionnelle à 25 s.

7.4 Lecture des résultats et limites

L’observation centrale est que **prob4** bénéficie de la marge étendue, et c’est précisément pour cette raison que la version rendue applique $\delta = 8$ côté L-SHADE. Cependant, plusieurs limites doivent être soulignées :

- Les scores du code rendu sur **prob4** (1562–1862) sont du même ordre que les meilleurs scores observés sur les moteurs isolés en *margin8*, mais ils restent supérieurs à 1 500 et la solution n’est pas pleinement faisable. Améliorer cette instance de façon garantie demanderait davantage de runs et probablement une graine plus informée géométriquement.
- Trois exécutions indépendantes ne suffisent pas pour conclure statistiquement : certains moteurs présentent des écarts entre runs supérieurs aux écarts entre algorithmes (par exemple CMA-ES **prob4** *margin8* : 1 517 vs 1 705). L-SHADE *margin8* fait exception sur **prob4** avec une dispersion d’environ 1 point, mais cela porte sur 3 runs uniquement.
- Les chiffres du code rendu et ceux des benchmarks historiques ne sont pas directement comparables : les premiers concernent l’hybridation complète, les seconds des moteurs isolés.

Le choix final `OptiPathFinal` privilégie donc la robustesse globale (faisable sur 7 instances sur 8), la conformité au cadre du sujet et la simplicité relative du code, plutôt que l’optimisation maximale d’une instance particulière.

8 Analyse

Constats positifs. L’hybridation parallèle s’est avérée robuste sur la plupart des instances. L-SHADE tire parti de ses bornes élargies pour autoriser des points de contrôle temporairement hors cadre, ce qui aide à franchir les passages étroits où une trajectoire purement intérieure est pénalisée par P_{bord} . CMA-ES, de son côté, bénéficie du pool de graines diversifiées et de son raffinement (1+1)-ES : lorsqu’une famille de graines positionne déjà la recherche dans un bassin raisonnable, les redémarrages IPOP en exploitent rapidement la covariance locale. La bascule conditionnelle à $t = 25$ s évite enfin de gaspiller du budget sur le moteur infaisable lorsque l’autre est déjà faisable. Quand les deux sont au même statut, aucun n’est désactivé.

Comportement par type d’instance. Les instances simples (peu d’obstacles, faible dimension) sont résolues de façon stable : la solution faisable est trouvée tôt par les deux moteurs, qui convergent ensuite vers une longueur de courbe quasi optimale. Sur les configurations à « murs » avec brèches, la grande population initiale de L-SHADE explore largement et certains individus traversent les ouvertures par chance. Sur les instances denses (par exemple `prob6`), la candidate A^* placée en tête du pool peut être retenue comme moyenne initiale de CMA-ES quand elle survit à la compétition, et la covariance s’aligne alors sur le couloir du labyrinthe. Quand elle est écartée au profit d’une graine sinusoïdale ou de bord, c’est typiquement parce que cette dernière obtient un meilleur score après (1+1)-ES.

Limite structurelle de l’évaluation. Sur `prob4` et plus généralement les labyrinthes denses, les deux moteurs peinent à atteindre la faisabilité dans le budget imparti. La difficulté n’est pas seulement combinatoire : `evaluateMulti` ne fournit pas un détecteur de collision local point par point, mais une pénalité agrégée sur l’ensemble de la courbe. Une correction géométrique naïve sur un point de contrôle peut donc améliorer la pénalité globale sans rendre la courbe faisable, et inversement. Cette agrégation explique aussi pourquoi une graine A^* discrète peut devenir infaisable une fois convertie en courbe de Bézier : les segments du polygone de contrôle ne coïncident pas avec la courbe, et l’inflation utilisée pour marquer les cellules bloquées ne garantit pas une marge équivalente sur la trajectoire continue. Le seeding ne peut donc pas être considéré comme un solveur géométrique. Il propose des amorces, et c’est CMA-ES (ou L-SHADE) qui doit ensuite ajuster les points de contrôle pour respecter la pénalité agrégée.

Rôle réel du polissage initial. La recherche locale (1+1)-ES appliquée aux 5 meilleures graines bénéficie au démarrage de CMA-ES, et uniquement à lui : L-SHADE est initialisé par tirage uniforme dans $[\ell, u]^d$ et ne reçoit aucune de ces graines. Le polissage réduit donc la variance du démarrage CMA-ES, mais n’a pas d’effet direct sur la trajectoire de L-SHADE.

Axes d’amélioration. Trois pistes restent ouvertes. Premièrement, certaines familles de graines (notamment les regroupements aux frontières) apportent peu sur les instances ouvertes et pourraient être conditionnées au profil détecté. Deuxièmement, la grille A^* est uniforme à 100×100 . Un raffinement adaptatif autour des obstacles denses pourrait fournir une candidate plus exploitable. Troisièmement, la bascule à $t = 25$ s ne tient compte que de la faisabilité : un critère prenant en compte la vitesse de convergence permettrait de réallouer le budget plus tôt sur les instances où l’un des moteurs est manifestement plus efficace.

9 Méthodes explorées mais non retenues

Avant de fixer la version rendue, une phase exploratoire a comparé plusieurs familles d’algorithmes et de variantes. Toutes ont été implémentées et benchmarkées, puis écartées au profit de l’hybridation L-SHADE + CMA-ES IPOPOP décrite dans les sections précédentes. Les paragraphes ci-dessous résument les familles, sans rentrer dans le détail quantitatif.

Variantes de CMA-ES. **BIPOP** et **NBIPOP** (alternance régimes large/small λ , allocation adaptative du budget entre régimes), **Active CMA-ES** (poids négatifs sur les pires individus, avec option de mémoire de covariance entre redémarrages), **Sep-CMA warmup** (phase diagonale $O(d)$ puis bascule sur la covariance pleine), et **LM-CMA** (approximation à mémoire limitée de Loshchilov 2014). Toutes ont été testées. Aucune n’a apporté un gain robuste suffisant pour justifier le surcoût en complexité ou en réglage. L’IPOPOP standard a été conservé pour sa simplicité et son comportement prévisible.

Modèles auxiliaires et gestion des contraintes. **Surrogates RBF** et **Random Forest** (pré-criblage de $\text{PRESCREENING_FACTOR} \times \lambda$ candidats par modèle bon marché, avec auto-désactivation par Kendall- τ), **Stochastic Ranking** (tri probabiliste équilibrant fitness et violation, Runarsson & Yao 2000), et **Lagrangien augmenté** (lissage du paysage par multiplicateurs λ). Le coût de mise à jour des modèles bon marché s’est révélé défavorable dans un budget de 60 s, et les schémas de gestion adaptative des contraintes ont eu un comportement moins stable que la pénalité fixe $\times 100$ déjà appliquée par le framework.

Portfolios et diversité. **Island model** (3 îles CMA-ES en round-robin avec migration en anneau), **Quality-Diversity / Map-Elites** (archive 2D indexée par descripteur comportemental, restart vers cellules peu visitées), et plusieurs **stratégies de restart exotiques** (frustration de pas, entropie d’occupation, accord de portfolio, plateau spectral, crédibilité bayésienne). Ces approches sont de bons outils d’exploration, mais leur coût en mono-thread ou leur instabilité sur les instances testées n’a pas justifié leur inclusion dans la version rendue.

Autres métaheuristiques de référence. **GA**, **PSO** (variantes **CLPSO**, **FPSO**), **Grey Wolf Optimizer** et **Firefly** ont été implémentés pour servir de références de comparaison. Ils ont été dominés par L-SHADE et CMA-ES sur les instances testées et n’apparaissent donc pas dans la version finale, mais ils ont confirmé le choix de la famille DE/CMA-ES pour ce problème.

Prototypes non conformes au cadre final. Plusieurs versions antérieures (**OptiPath**, **OptiPathMk2** à **Mk5_3**) accédaient directement aux obstacles via réflexion sur le champ privé `obstacles` de `Problem`, lisaient des fichiers CSV externes contenant des solutions de référence, ou se spécialisaient selon le nom de l’instance. Ces prototypes ont été utiles mais ils ont été exclus de la version rendue car ils ne respectent pas les règles du sujet. La version rendue n’utilise que `evaluate` et `evaluateMulti`, ne lit aucun fichier externe, et ne contient aucune branche conditionnée par le nom de l’instance.

10 Conclusion

OptiPathFinal combine L-SHADE (bornes élargies) et CMA-ES IPOPOP (moyenne initiale sélectionnée au sein d’un pool de graines diversifiées incluant une candidate A^*) exécutés en parallèle, avec bascule conditionnelle à 25 s. La version rendue est faisable sur 7 instances sur 8. **prob4** reste partiellement infaisable. Les limites (lancement unique sur le rendu, 3 runs sur les benchmarks historiques) et trois pistes d’amélioration (familles de graines conditionnelles, grille A^* adaptative, critère de bascule fondé sur la vitesse de convergence) sont détaillées en §8.

Références

- [1] R. Storn et K. Price, “Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces,” *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.
- [2] R. Tanabe et A. Fukunaga, “Success-history based parameter adaptation for Differential Evolution,” *Proc. IEEE Congress on Evolutionary Computation (CEC)*, pp. 71–78, 2013.
- [3] R. Tanabe et A. Fukunaga, “Improving the search performance of SHADE using linear population size reduction,” *Proc. IEEE Congress on Evolutionary Computation (CEC)*, pp. 1658–1665, 2014.
- [4] N. Hansen et A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [5] N. Hansen, “The CMA Evolution Strategy : A Tutorial,” arXiv :1604.00772, 2016.
- [6] A. Auger et N. Hansen, “A restart CMA evolution strategy with increasing population size,” *Proc. IEEE Congress on Evolutionary Computation (CEC)*, pp. 1769–1776, 2005.
- [7] P. E. Hart, N. J. Nilsson et B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.